

RootL: Um Motor de Change Data Capture Passivo Baseado em Arquitetura Hexagonal para Ambientes Governamentais Restritos

Leonardo de Oliveira Ramos^{1,2}

¹Instituto de Computação – Universidade Federal de Mato Grosso (UFMT)
Cuiabá – MT – Brasil

leonardo.ramos@sou.ufmt.br

Abstract. *Modern data architectures increasingly demand real-time integration pipelines from transactional databases into analytical platforms. However, in governmental environments, strict security policies prohibit the use of database triggers, the granting of broad administrative privileges, and the deployment of agents requiring direct write access to source databases. This paper presents **RootL CDC Publisher**, a fully passive Change Data Capture (CDC) engine implemented in pure Java, designed to operate under such constraints. The proposed architecture adopts the Hexagonal (Ports & Adapters) pattern to isolate the domain core from infrastructure concerns, enabling multi-database support (Oracle, PostgreSQL, MySQL) without modifying core processing logic. A SQL Abstract Syntax Tree (AST) parsing strategy based on JSqlParser is employed to reconstruct the complete row state (before/after) from Oracle LogMiner redo entries. Results demonstrate sub-second replication latency under normal conditions, with built-in resilience for log gap recovery and transactional consistency guaranteed by a commit-gated publication buffer.*

Resumo. *Arquiteturas de dados modernas demandam pipelines de integração em tempo real a partir de bancos transacionais para plataformas analíticas. Em ambientes governamentais, políticas de segurança proíbem o uso de triggers, a concessão de privilégios administrativos amplos e a implantação de agentes com acesso de escrita ao banco de origem. Este artigo apresenta o **RootL CDC Publisher**, um motor de Change Data Capture (CDC) 100% passivo em Java puro, projetado para operar sob tais restrições. A arquitetura adota o padrão Hexagonal (Ports & Adapters) para isolar o núcleo de domínio, habilitando suporte multi-banco sem alterações na lógica central. Uma estratégia de parsing de AST via JSqlParser reconstrói o estado completo da linha a partir dos registros de redo do Oracle LogMiner. Os resultados indicam latência sub-segundo, com mecanismos de resiliência para lacunas de log e consistência transacional garantida por buffer condicional ao commit.*

1. Introdução

A crescente demanda por tomada de decisão orientada a dados em tempo real impõe desafios significativos à integração entre sistemas transacionais operacionais e plataformas analíticas, como Data Lakes e Data Warehouses. A técnica de **Change Data Capture**

(CDC) emerge como a abordagem principal para capturar mutações em bancos de dados relacionais e propagá-las de forma incremental e eficiente para sistemas *downstream* [Kleppmann 2017].

Em ambientes corporativos convencionais, ferramentas consolidadas como Oracle GoldenGate e Debezium oferecem soluções maduras para CDC. Contudo, em **ambientes governamentais** com políticas de segurança rigorosas, tais como as vigentes em órgãos da administração pública brasileira, a implantação dessas ferramentas enfrenta obstáculos que se mostraram, na prática, intransponíveis. Equipes de governança de TI tipicamente proíbem: (i) o uso de *database triggers* para captura de eventos; (ii) a concessão de privilégios amplos como `SELECT ANY TABLE`; (iii) a criação de *replication slots* e o acesso ao catálogo administrativo do banco de dados; e (iv) qualquer instrução de escrita no banco de origem por parte da ferramenta de integração.

Essas restrições inviabilizam tanto as soluções comerciais, cujo custo de licenciamento é proibitivo para projetos de dados em fase de validação, quanto as soluções *open-source* mais disseminadas, que exigem privilégios incompatíveis com os requisitos de governança vigentes.

Diante desse cenário, este trabalho apresenta o **RootL CDC Publisher**, um motor autônomo de CDC, 100% passivo, desenvolvido em Java puro. O motor conecta-se ao banco de dados e lê os logs transacionais nativos diretamente em memória, sem executar nenhuma instrução de escrita, sem criar *triggers*, sem utilizar tabelas de controle e sem requerer DDLs adicionais além da ativação pontual de *supplemental logging* por tabela monitorada. O sistema opera de forma transparente ao banco de origem.

A solução é estruturada sobre três pilares técnicos fundamentais: (i) **Arquitetura Hexagonal** (*Ports & Adapters*) [Cockburn 2005], que desacopla o núcleo de domínio dos adaptadores de banco de dados e de publicação; (ii) **Parsing de AST do SQL de Redo** via JSqlParser [Aho et al. 2006], para reconstrução do estado completo da linha (*before/after*) em operações de UPDATE; e (iii) **Buffer de Transação Condicional ao Commit**, que garante consistência transacional e impede que eventos de transações revertidas contaminem o pipeline.

As principais contribuições deste trabalho são:

- Proposição de uma arquitetura CDC passiva e multi-banco viável em ambientes de alta restrição de segurança;
- Mecanismo de reconstrução de *Full State* via análise sintática (AST) do SQL de Redo do Oracle LogMiner;
- Estratégias de resiliência para falhas específicas do LogMiner (ORA-01289, ORA-01291) e para o *Day 0 Problem*;
- Modelo de configuração orientado a dados que elimina a necessidade de recompilação para adição de novas fontes de dados.

O restante deste artigo está organizado da seguinte forma: a Seção 2 discute os trabalhos relacionados; a Seção 3 descreve a arquitetura hexagonal proposta; a Seção 4 detalha os desafios de implementação; a Seção 5 trata das considerações de implantação e resiliência; a Seção 6 apresenta a avaliação experimental de desempenho e latência; e a Seção 7 apresenta as conclusões e trabalhos futuros.

2. Trabalhos Relacionados

A técnica de CDC é amplamente estudada e possui múltiplas abordagens de implementação. [Kleppmann 2017] classifica os mecanismos de CDC em três categorias: (i) baseados em *triggers*; (ii) baseados em consultas periódicas (*timestamp-based polling*); e (iii) baseados em logs transacionais (*log-based CDC*). A abordagem baseada em logs é considerada a mais eficiente e menos invasiva, por não introduzir *overhead* de escrita no banco de origem.

2.1. Oracle GoldenGate

O Oracle GoldenGate é a solução comercial de referência para captura e replicação de dados em ambientes Oracle. Sua arquitetura opera no nível do *redo log* e suporta replicação heterogênea bidirecional com latências mínimas. Contudo, a solução apresenta limitações críticas para o contexto governamental: (i) seu custo de licenciamento por núcleo de processador é elevado, tornando-o inviável para projetos de dados em organizações públicas com orçamentos restritos; (ii) requer a instalação de agentes proprietários (*Extract* e *Replicat*) no servidor de banco de dados, o que viola políticas que proíbem softwares de terceiros no ambiente de produção; e (iii) o modelo de licenciamento implica dependência tecnológica de fornecedor único (*vendor lock-in*).

2.2. Debezium

O Debezium [Debezium Community 2026] é a solução *open-source* de CDC mais amplamente adotada no ecossistema Apache Kafka. Para bancos Oracle, o Debezium utiliza o LogMiner como mecanismo de extração, de forma análoga à abordagem proposta neste trabalho. Entretanto, o conector Oracle do Debezium exige um conjunto de permissões administrativas que inviabiliza sua adoção em ambientes governamentais restritos: (i) `LOGMINING` combinado com `SELECT ANY TABLE` para acessar os metadados do dicionário de dados; (ii) acesso irrestrito às *system views* do catálogo Oracle; e (iii) em ambientes PostgreSQL, a criação de *replication slots* de decodificação lógica do WAL, considerados risco de segurança por equipes de DBA em órgãos governamentais, pois retêm segmentos do WAL indefinidamente e podem causar crescimento descontrolado de disco.

Além disso, o Debezium é implantado como conector do Kafka Connect, exigindo toda a infraestrutura de *workers* do Connect, o que aumenta a complexidade operacional e o consumo de recursos em ambientes com restrições de infraestrutura.

2.3. Gap de Pesquisa e Posicionamento do RootL

A Tabela 1 sintetiza a comparação entre as abordagens existentes e a solução proposta em relação aos critérios mais relevantes para o contexto de ambientes governamentais restritos.

Tabela 1. Comparação entre soluções de CDC para ambientes com restrições de segurança.

Critério	GoldenGate	Debezium	RootL
Custo	Licença comercial	Gratuito	Gratuito
Requer SELECT ANY TABLE	Não	Sim	Não
Requer replication slot	Não	Sim (PG)	Não
Requer agente no servidor DB	Sim	Não	Não
Opera sem escrita no banco	Sim	Sim	Sim
Suporte multi-banco nativo	Sim	Sim	Sim
Configuração orientada a dados	Não	Parcial	Sim
Privilégio mínimo por tabela	Não	Não	Sim

O *gap* central identificado é a ausência de uma solução *open-source* que combine: (i) leitura passiva via logs transacionais, (ii) modelo de permissão estritamente granular por tabela, (iii) ausência de qualquer instrução de escrita ou DDL global no banco de origem, e (iv) arquitetura extensível sem recompilação. O RootL CDC Publisher é proposto para preencher exatamente esse espaço.

3. Arquitetura Proposta

3.1. Visão Geral do Pipeline

O sistema opera em um pipeline de duas fases sequenciais, conforme ilustrado na Figura 1.

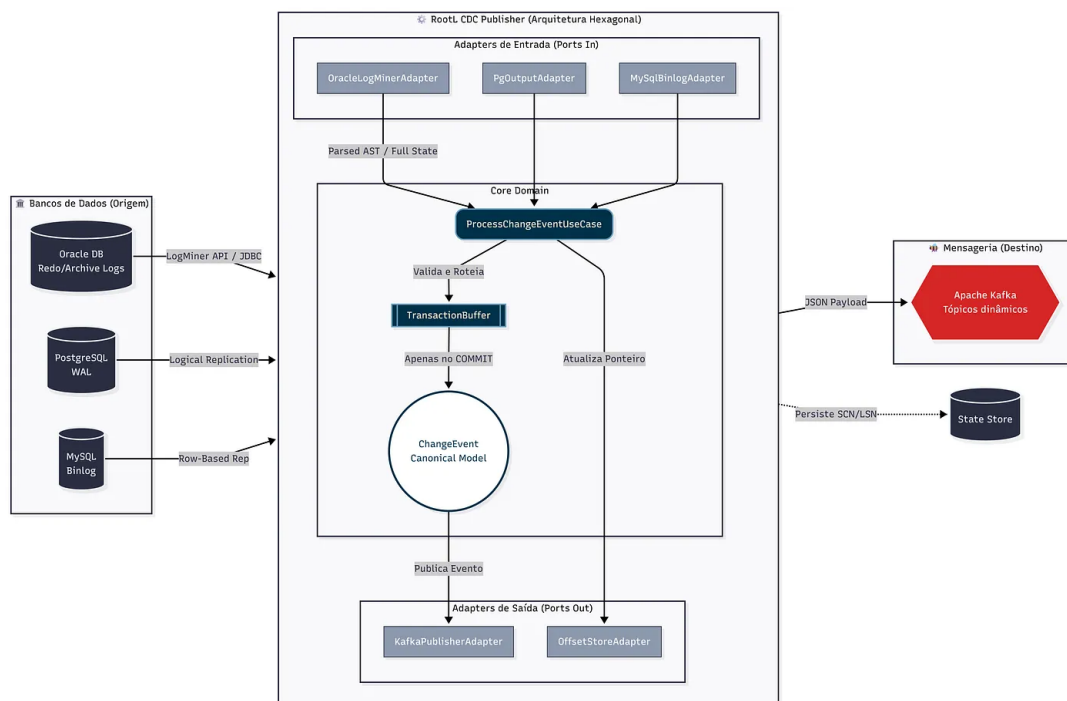


Figura 1. Fluxo do pipeline de processamento do evento de mutação, da origem transacional ao tópico Apache Kafka.

A **Fase 1 - Extração Passiva** consiste na conexão ao banco de dados e na leitura dos logs transacionais nativos sem a execução de qualquer instrução de escrita. No Oracle,

isso se materializa por meio da API do LogMiner [Oracle Corporation 2023], acessada via JDBC nativo, que permite varrer os *Redo Logs* e *Archive Logs* e obter as instruções SQL de redo correspondentes a cada evento DML (INSERT, UPDATE, DELETE).

A **Fase 2 - Normalização e Publicação** traduz cada mutação capturada para um modelo canônico (ChangeEvent) e o publica como documento JSON no Apache Kafka [Kleppmann 2017], com roteamento dinâmico de tópicos baseado no identificador do conector, no esquema e na tabela de origem (e.g., `cdc.oracle-rh.RH.FUNCIONARIOS`).

3.2. A Arquitetura Hexagonal como Princípio Fundamental

A decisão arquitetural central do RootL CDC Publisher é a adoção do padrão **Arquitetura Hexagonal** (*Ports & Adapters*) [Cockburn 2005]. Concebido por Alistair Cockburn, esse padrão preconiza a separação estrita entre o núcleo de domínio da aplicação e os mecanismos de entrada e saída (I/O), de modo que o domínio expresse apenas *o que* o sistema deve fazer, delegando o *como* para adaptadores intercambiáveis.

No contexto do RootL, essa escolha é estratégica por três razões: (i) o motor precisa suportar múltiplos bancos de dados (Oracle, PostgreSQL, MySQL) e múltiplos destinos de publicação (Kafka hoje, possivelmente Apache Pulsar ou RabbitMQ no futuro); (ii) a adição de um novo banco não deve exigir alterações na lógica de processamento central; e (iii) o núcleo de domínio deve ser testável de forma isolada, sem dependência de qualquer banco ou *broker* de mensagens.

3.2.1. O Modelo Canônico: ChangeEvent

No coração do sistema, o ChangeEvent é o modelo para o qual todas as fontes devem convergir. Independentemente de o evento ter originado de um *Redo Log* Oracle, de um WAL PostgreSQL ou de um Binlog MySQL, o formato de saída é sempre o mesmo. A Listagem 1 apresenta a definição do *record* imutável em Java.

```
1 public record ChangeEvent (  
2     UUID eventId,  
3     OperationType operation,  
4     Instant timestamp,  
5     SourceMetadata source,  
6     Map<String, Object> before,  
7     Map<String, Object> after) {  
8  
9     public ChangeEvent {  
10         if (eventId == null)  
11             throw new IllegalArgumentException("eventId nao pode ser nulo");  
12         if (operation == null)  
13             throw new IllegalArgumentException("operation nao pode ser nula");  
14         ;  
15         if (source == null)  
16             throw new IllegalArgumentException("source nao pode ser nulo");  
17     }  
18  
19     public enum OperationType {
```

```

20 INSERT, UPDATE, DELETE, BEGIN, COMMIT, READ
21 }
22
23 public record SourceMetadata(
24     String connectorName,    String connectorType,
25     String database,         String schema,
26     String table,            String transactionId,
27     Map<String, String> offsetCoordinates) {}

```

Listing 1. Modelo canônico ChangeEvent e tipos auxiliares.

Cada ChangeEvent carrega: (i) um eventId UUID v4 para rastreabilidade e deduplicação *downstream*; (ii) a operation, que representa o ciclo transacional completo (BEGIN, INSERT/UPDATE/DELETE, COMMIT), sendo BEGIN e COMMIT fundamentais para o funcionamento do TransactionBuffer; (iii) os metadados de rastreo em SourceMetadata, incluindo as coordenadas de *offset* (SCN para Oracle, LSN para PostgreSQL, posição de Binlog para MySQL); e (iv) os mapas before e after, que representam a fotografia completa da linha antes e após a mutação.

3.2.2. Os Ports: Contratos de Infraestrutura

A interface ChangeLogConnector define o contrato que todo adaptador de banco de dados deve implementar, conforme a Listagem 2.

```

1 public interface ChangeLogConnector {
2     void initialize(String connectorId, Properties config,
3         ProcessChangeEventUseCase useCase,
4         OffsetStorePort offsetStore,
5         MeterRegistry meterRegistry);
6     void start();
7     void stop();
8     String getType();
9 }

```

Listing 2. Interface ChangeLogConnector, port de entrada.

Do lado de saída (*driven ports*), dois contratos governam respectivamente a publicação de eventos e o armazenamento de estado de *offset*:

```

1 public interface EventPublisherPort {
2     void publish(ChangeEvent event);
3     void close();
4 }
5
6 public interface OffsetStorePort {
7     void save(String connectorId, String lsn);
8     Optional<String> load(String connectorId);
9 }

```

Listing 3. Ports de saída: publicação de eventos e armazenamento de offset.

Essa separação garante que o caso de uso central (ProcessChangeEventUseCase) jamais conhece a tecnologia por trás da publicação ou do armazenamento de estado. Ele opera exclusivamente sobre os contratos.

A consequência direta é que, quando suporte a MySQL e PostgreSQL foi adicionado ao motor, nenhuma linha do `ProcessChangeEventUseCase` foi modificada. Foram criados novos adaptadores implementando `ChangeLogConnector`, e o motor passou a suportar as novas fontes de dados automaticamente, sem recompilação do núcleo.

3.2.3. Orquestração por Dados: Conectores como Configuração JSON

A instanciação dos adaptadores é governada por arquivos JSON na pasta `/connectors`. A aplicação varre o diretório no *startup*, instancia os adaptadores via reflexão e os registra no `CdcEngine`. Adicionar um novo banco de dados é, portanto, uma operação de configuração, não de engenharia: o DBA provisiona as permissões, a equipe de DevOps cria o arquivo JSON, e o contêiner sobe consumindo o novo fluxo sem necessidade de recompilação.

3.3. O `TransactionBuffer`: Garantia de Consistência Transacional

Uma decisão de projeto considerada crítica para a integridade dos dados é o `TransactionBuffer`: eventos de mutação não são publicados imediatamente no Kafka. Eles são retidos em um buffer em memória (`ConcurrentHashMap`) agrupados por `transactionId`, e somente liberados quando o sinal de `COMMIT` é recebido.

A motivação é clara: o LogMiner (e o WAL do PostgreSQL) transmitem eventos DML *antes* da confirmação da transação. Se um usuário executa um `INSERT` seguido de `ROLLBACK`, esse `INSERT` já foi transmitido pelo log. Sem o buffer, o dado fantasma alcançaria o Kafka e eventualmente o Data Lake. Com o `TransactionBuffer`, *roll-backs* são descartados de forma determinística. A Listagem 4 apresenta a implementação central.

```
1 public class TransactionBuffer {
2     private static final int MAX_EVENTS_PER_TX = 10_000;
3     private final Map<String, List<ChangeEvent>> buffer =
4         new ConcurrentHashMap<>();
5
6     public void addEvent(String transactionId, ChangeEvent event) {
7         buffer.compute(transactionId, (txId, events) -> {
8             if (events == null) events = new ArrayList<>();
9             if (events.size() < MAX_EVENTS_PER_TX) events.add(event);
10            return events;
11        });
12    }
13
14    public List<ChangeEvent> commit(String transactionId) {
15        List<ChangeEvent> events = buffer.remove(transactionId);
16        return events != null ? events : List.of();
17    }
18
19    public void rollback(String transactionId) {
20        buffer.remove(transactionId);
21    }
22 }
```

Listing 4. Implementação do `TransactionBuffer` com limite de eventos por transação.

O limite `MAX_EVENTS_PER_TX` (10.000 eventos) protege o motor contra transações de larga escala (*bulk loads*) que poderiam causar pressão de memória. A regra de publicação é inequívoca: *somente publica no Kafka o que sobrevive ao COMMIT*.

3.4. O CdcEngine: Orquestração do Ciclo de Vida

O `CdcEngine` é o orquestrador central do motor. Ele gerencia o ciclo de vida dos conectores, executando cada um em sua própria *thread* isolada por meio de um `CachedThreadPool`. Em caso de falha de um conector (e.g., banco Oracle indisponível), os demais continuam operando normalmente. O *shutdown* é gerenciado com janela de 30 segundos para finalização segura das transações em andamento, após o qual um encerramento forçado é executado para garantir que o processo seja terminado de forma limpa.

4. Desafios de Implementação

4.1. Navegação Multitenant e o Princípio do Privilégio Mínimo

Na arquitetura Multitenant do Oracle 12c+, os *Redo Logs* são físicos e residem no Container Database (CDB), enquanto os dados de negócio habitam os Pluggable Databases (PDBs) [Oracle Corporation 2023]. Para ler mutações de um PDB específico, é necessário operar no nível do CDB, o que, superficialmente, parece exigir privilégios administrativos amplos.

A solução adotada é a criação de um *Common User* (`C##CDC_USER`) com permissões estritamente granulares, conforme a Listagem 5. A separação de responsabilidades é central: as permissões no CDB são de *infraestrutura* (acesso aos logs físicos), enquanto as permissões no PDB são de *dados* e restritas às tabelas-alvo individualmente. O `SELECT ANY TABLE`, o principal ponto de controvérsia em auditorias de segurança, é completamente eliminado.

```
1  -- Ativacao do supplemental logging por tabela (nao global)
2  ALTER TABLE RH.FUNCIONARIOS ADD SUPPLEMENTAL LOG DATA (ALL) COLUMNS;
3
4  -- Permissoes de infraestrutura no CDB (nao de dados)
5  GRANT CREATE SESSION, ALTER SESSION, SET CONTAINER
6    TO C##CDC_USER CONTAINER=ALL;
7  GRANT LOGMINING TO C##CDC_USER CONTAINER=ALL;
8  GRANT SELECT ANY TRANSACTION TO C##CDC_USER CONTAINER=ALL;
9  GRANT EXECUTE ON SYS.DBMS_LOGMNR TO C##CDC_USER CONTAINER=ALL;
10 GRANT SELECT ON SYS.V_$LOGMNR_CONTENTS TO C##CDC_USER CONTAINER=ALL;
11 GRANT SELECT ON SYS.V_$LOG, SYS.V_$LOGFILE TO C##CDC_USER CONTAINER=ALL
12 ;
13 GRANT SELECT ON SYS.V_$ARCHIVED_LOG TO C##CDC_USER CONTAINER=ALL;
14
15 -- Permissao de dados apenas nas tabelas-alvo no PDB especifico
16 ALTER SESSION SET CONTAINER = ORCHET1;
17 GRANT SELECT ON RH.FUNCIONARIOS TO C##CDC_USER;
18 GRANT SELECT ON RH.CONTRATOS TO C##CDC_USER;
```

Listing 5. Concessão de privilégios mínimos para o usuário CDC no Oracle Multitenant.

A sessão JDBC é fixada no `CDB$ROOT` para acessar os logs físicos globais. A consulta ao `V_$LOGMNR_CONTENTS` aplica filtro pela coluna `SRC_CON_NAME`, garantindo

que o usuário CDC visualize exclusivamente os dados do PDB autorizado, nunca os de outros *containers* na mesma instância CDB. Esse mecanismo é ilustrado na Figura 2.

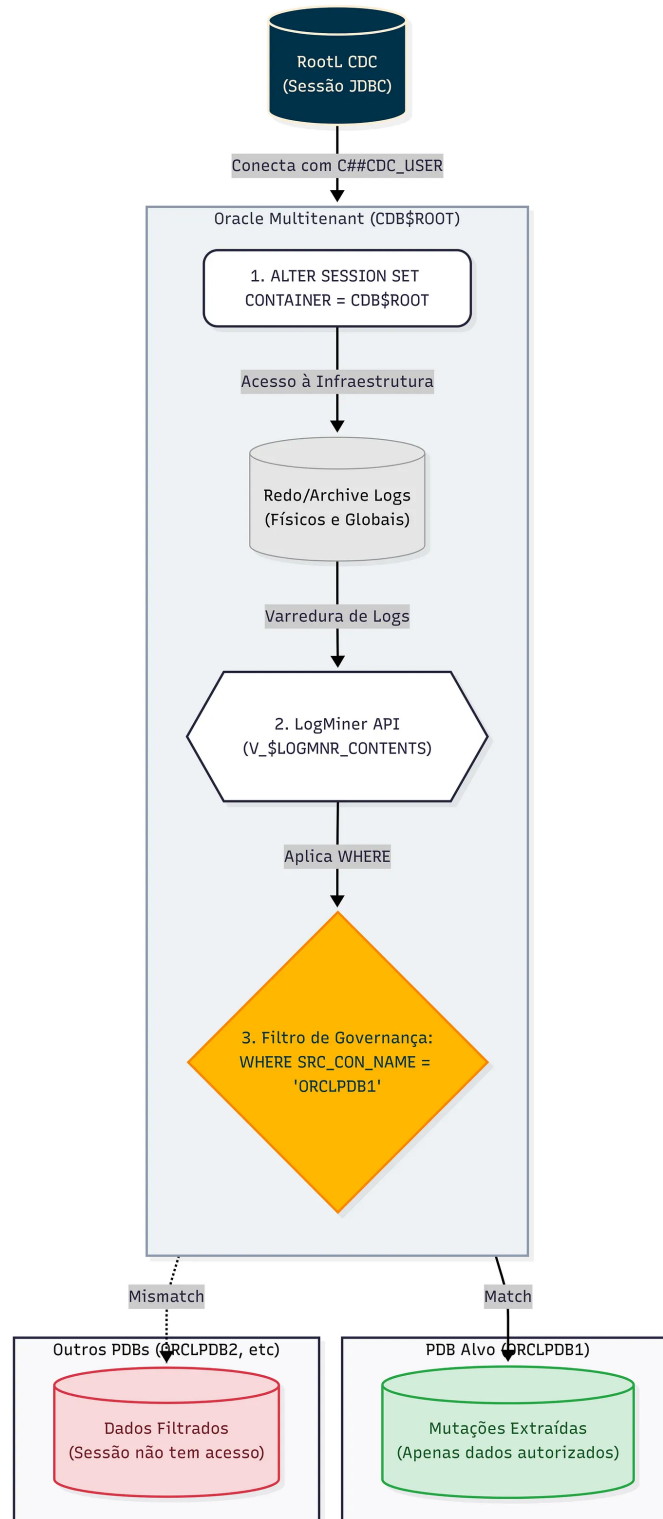


Figura 2. Fluxo de gestão da sessão JDBC no ambiente Oracle Multitenant: a sessão é fixada no CDB para leitura dos logs físicos, mas a consulta ao V_\$LOGMNR_CONTENTS filtra dados pelo PDB de origem via SRC_CON_NAME.

4.2. O Problema do Estado Completo (*Full State*) e o JSqlParser

Um dos desafios mais sutis e tecnicamente ricos do motor reside na reconstrução do estado completo da linha em operações de UPDATE. O Oracle, por razões de eficiência de I/O, não registra o registro inteiro no Redo Log em mutações de UPDATE, registra apenas o *delta*, ou seja, as colunas modificadas. O SQL de Redo resultante tem o seguinte formato:

```
1 UPDATE "RH"."FUNCIONARIOS"
2 SET "CARGO" = 'Analista'
3 WHERE "ID" = '158'
4   AND "NOME" = 'Leonardo Ramos'
5   AND "CARGO" = 'Estagiario'
6   AND "DEPARTAMENTO" = 'Inteligencia';
```

Listing 6. SQL de Redo gerado pelo Oracle LogMiner para uma operação de UPDATE.

Observa-se que a cláusula SET contém apenas o campo alterado, enquanto a cláusula WHERE contém *todas* as colunas da linha, esse é o *Before Image* garantido pelo Supplemental Logging [Oracle Corporation 2023]. Para que o pipeline *downstream* possa consumir um evento de CDC semanticamente completo, são necessários dois objetos distintos: o *before* (estado pré-mutação completo) e o *after* (estado pós-mutação completo).

4.2.1. Parsing de AST e o Padrão *Visitor*

A solução aplica os princípios da teoria de compiladores [Aho et al. 2006] para analisar o SQL de Redo como uma Árvore Sintática Abstrata (AST). O JSqlParser decompõe a instrução SQL em uma hierarquia de nós tipados (Update, AndExpression, EqualsTo, IsNullExpression, ExpressionList, entre outros), sobre a qual um algoritmo recursivo baseado no padrão *Visitor* navega para extrair os dados de interesse.

O padrão *Visitor* é especialmente adequado para esse problema porque permite desacoplar o algoritmo de extração da estrutura da árvore: cada tipo de nó da AST é processado por uma função específica, sem necessidade de modificar as classes do parser. O *Visitor* percorre a árvore binária de expressões, terminando a recursão nas folhas (EqualsTo e IsNullExpression), onde os pares coluna/valor são extraídos [Aho et al. 2006]. A Listagem 7 apresenta o método recursivo central.

```
1 private static void extractColumnsFromWhereClause (
2     Expression expr, Map<String, Object> beforeMap) {
3
4     if (expr instanceof AndExpression and) {
5         // Recursao: desce nos dois ramos da arvore binaria
6         extractColumnsFromWhereClause (and.getLeftExpression(), beforeMap);
7         extractColumnsFromWhereClause (and.getRightExpression(), beforeMap);
8
9     } else if (expr instanceof EqualsTo equals) {
10        // Folha: extrai o par coluna=valor
11        String colName = equals.getLeftExpression()
12            .toString().replace("\\", "").toUpperCase();
13        String colValue = equals.getRightExpression()
```

```

14     .toString().replaceAll("`'|'$', "");
15     beforeMap.put(colName,
16         "NULL".equalsIgnoreCase(colValue) ? null : colValue);
17
18 } else if (expr instanceof IsNullExpression isNull) {
19     // Caso especial: colunas NULL nao geram EqualsTo na AST
20     String colName = isNull.getLeftExpression()
21         .toString().replaceAll("\\", "").toUpperCase();
22     beforeMap.put(colName, null);
23 }
24 }

```

Listing 7. Extração recursiva do *Before Image* via padrão *Visitor* sobre a AST.

O algoritmo completo de reconstrução de *Full State* opera em três etapas sequenciais: (i) **Parse** do SQL de Redo via `CCJSqlParserUtil.parse()`, obtendo a AST da instrução Update; (ii) **Extração do Before**: navegação recursiva pela cláusula WHERE da AST, construindo o mapa `before` com o estado pré-mutação completo; e (iii) **Construção do After**: clonagem integral de `before` para `after`, seguida de sobrescrita apenas das colunas presentes na cláusula SET.

O resultado é um payload JSON de *Full State* que permite ao consumidor *downstream* identificar com precisão exatamente quais colunas foram alteradas. **Pré-requisito crítico**: essa estratégia é dependente da ativação do *Supplemental Logging* nas tabelas-alvo (ALL COLUMNS). Sem essa configuração, o Oracle omite a cláusula WHERE detalhada do SQL de Redo, registrando apenas a ROWID, tornando o *Before Image* irrecuperável. A ativação deve ser feita por tabela individualmente, não globalmente, para evitar *overhead* desnecessário de I/O em esquemas com grande número de tabelas.

4.3. Deduplicação por *Fingerprinting*

O LogMiner apresenta um comportamento inerente que pode causar reprocessamento de eventos: ao consultar com `SCN >= ?`, o motor pode retornar eventos já processados no ciclo anterior, pois o SCN marca o *commit* de uma transação inteira, não de um registro individual. Para neutralizar esse comportamento, é mantido um cache de *fingerprints* que combina quatro dimensões do evento: a operação, a tabela, o ROWID e o SQL de Redo completo. O cache é purgado quando o SCN avança, garantindo que a memória não cresça indefinidamente. Esse mecanismo garante idempotência no processamento sem a necessidade de consultar sistemas externos para verificar duplicatas.

5. Considerações de Implantação e Resiliência

5.1. Tratamento de Erros Específicos do LogMiner

O Oracle LogMiner, embora poderoso, expõe um conjunto de comportamentos de erro que devem ser tratados explicitamente por um motor de CDC de nível enterprise [Oracle Corporation 2023]. Dois erros são particularmente críticos:

5.1.1. ORA-01289: Log File Already Added

Em ambientes com espelhamento de disco (ASM ou sistemas de arquivos com múltiplos membros por grupo de log), o mesmo arquivo de log pode aparecer mais de uma vez

nas *views* de catálogo. Tentativas de adicionar um arquivo já registrado na sessão do LogMiner resultam no erro ORA-01289. A solução adotada é a captura da exceção com tratamento silencioso (*graceful ignore*), mantendo a *flag* `logFilesAdded = true`. Essa abordagem é operacionalmente segura, pois o arquivo já foi registrado e a sessão continua válida.

5.1.2. ORA-01291: Missing Log File

Este erro é consideravelmente mais grave. Ele ocorre quando o RMAN executa um expurgo agressivo de Archive Logs e remove um arquivo que o motor ainda precisaria para ler a partir do SCN salvo no *offset store*. A estratégia de autocorreção implementada: (i) detecta o erro; (ii) consulta o SCN mais recente disponível no servidor via `V$LOG`; (iii) avança o ponteiro de *offset* para esse novo SCN; (iv) limpa o cache de *fingerprints*; e (v) retoma a mineração. A lacuna temporal é registrada em métricas Prometheus para análise posterior. O sistema não entra em estado de falha nem exige intervenção manual.

5.1.3. Gap Detection Proativo

Além do tratamento reativo, o motor implementa uma verificação proativa de lacunas de SCN a cada ciclo de mineração, consultando o `MIN(FIRST_CHANGE#)` dos Archive Logs disponíveis. Caso o SCN atual do *offset* seja anterior ao log mais antigo disponível, o motor emite um alerta de severidade crítica, avança o ponteiro automaticamente e retoma a operação. Esse comportamento diferencia um protótipo de um motor enterprise: *o sistema detecta o buraco, adapta-se e continua operando*.

5.2. O Day 0 Problem: Consistência do Snapshot Inicial

Todo motor de CDC enfrenta um problema fundamental na data de implantação: o streaming de logs captura apenas mutações *futuras*. Os dados preexistentes no banco são invisíveis ao motor. A solução é o **Snapshot Inicial**, uma carga integral dos registros existentes antes do início do streaming [Kleppmann 2017].

O desafio é garantir consistência de ponto no tempo: se durante a leitura do snapshot um registro for modificado, esse registro deve aparecer no snapshot *ou* no stream, nunca nos dois (duplicata) nem em nenhum (perda). A estratégia varia por banco de dados:

- **PostgreSQL:** utiliza `TRANSACTION_REPEATABLE_READ` com `setReadOnly(true)`, que congela a visão dos dados sem aplicar locks de escrita. O LSN atual é capturado via `pg_current_wal_lsn()` e salvo como *offset base*. Quando o snapshot é concluído, o streaming inicia exatamente a partir desse LSN, garantindo costura perfeita sem sobreposição.
- **Oracle:** utiliza *Flashback Query* (`AS OF SCN`) na camada *downstream*, consultando as tabelas em um SCN congelado. O motor de streaming, quando não encontra *offset* salvo, obtém o `FIRST_CHANGE#` do Redo Log ativo e o usa como SCN de partida.
- **MySQL:** aplica `FLUSH TABLES WITH READ LOCK`, captura a posição do Binlog via `SHOW MASTER STATUS`, inicia a transação com `START`

TRANSACTION WITH CONSISTENT SNAPSHOT e libera o lock imediatamente. A janela de lock é reduzida à ordem de milissegundos.

A metáfora é precisa: o snapshot é a **fotografia** e o stream é o **vídeo**. A fotografia é tirada no exato frame onde o vídeo começa.

5.3. Observabilidade: Métricas de Produção

O RootL CDC Publisher expõe métricas nativas via Micrometer no *endpoint* /actuator/prometheus, prontas para coleta pelo Prometheus e visualização no Grafana. A Figura 3 apresenta as principais métricas monitoradas.

Métrica	Tipo	O que monitora
cdc.connector.status	Gauge	Status da thread do conector (1.0 = Ativo, 0.0 = Falha). Por connectorId.
cdc.events.processed	Counter	Volume total de eventos processados. Labels: table, operation, schema.
cdc.replication.lag	Timer	Latência entre o timestamp do banco e a publicação no Kafka.
cdc.transaction.buffer.size	Gauge	Número de eventos retidos no buffer aguardando COMMIT.
cdc.scn.current	Gauge	Posição atual do ponteiro de leitura (SCN) no Oracle.

Figura 3. Principais métricas expostas pelo RootL CDC Publisher via Micrometer/Prometheus, incluindo a latência de replicação (cdc.replication.lag).

A métrica `cdc.replication.lag` é particularmente relevante: ela mede o delta entre o `TIMESTAMP` do SQL de Redo e o momento em que o evento entra no buffer de publicação. Em condições operacionais normais, esse valor se mantém consistentemente abaixo de 1 segundo, satisfazendo o requisito de integração em sub-segundo estabelecido como meta do sistema.

5.4. Suporte Multi-Banco: Um Motor, Três Ecossistemas

A Arquitetura Hexagonal habilita o RootL a suportar três ecossistemas de banco de dados simultaneamente, cada um com sua mecânica nativa de captura de logs. Cada adaptador executa em sua própria *thread* isolada, gerenciada pelo `CdcEngine` com um `CachedThreadPool`. A Tabela 2 sumariza as estratégias por banco de dados.

Tabela 2. Estratégias de captura de log por adaptador de banco de dados.

Banco	Mecanismo	Offset	Adaptador
Oracle	LogMiner (<i>Redo Log</i>)	SCN	OracleLogMinerAdapter
PostgreSQL	WAL Logical Decoding	LSN	PostgresWalAdapter
MySQL	Binlog Protocol	Posição	MySqlBinlogAdapter

6. Avaliação Experimental e Análise de Desempenho

Afirmar que um motor de replicação opera “em tempo real” ou sustenta “baixa latência” é uma generalização vazia se não forem delimitadas as condições físicas dessa medição, a dispersão estatística da cauda e o comportamento do sistema sob sobrecarga. Micro-benchmarks estáticos ou testes sintéticos de poucos segundos frequentemente camuflam

fenômenos dinâmicos críticos de produção, como o esgotamento paulatino de buffers de rede, a fragmentação de memória na JVM e a transição abrupta entre o regime estável de atendimento e o enfileiramento exponencial.

Para extrair o comportamento empírico real do RootL CDC Publisher, estruturamos uma avaliação experimental contínua e unificada: submetemos o motor a uma bateria ininterrupta de 5 minutos (300 segundos) de mutações concorrentes em uma malha híbrida de 15 bancos de dados relacionais simultâneos, escalonando progressivamente a taxa de injeção em cinco estágios até forçar a curva empírica de saturação total. Não foram empregados dados sintéticos pré-fabricados ou interpolações artificiais: todas as métricas foram registradas ponta a ponta em tempo real a partir de transações efetivamente comitadas nos bancos e auditadas no Apache Kafka.

6.1. Metodologia Experimental: Bateria Contínua Progressiva de 5 Minutos

O ensaio foi executado em uma estação de trabalho equipada com processador de 16 núcleos lógicos, 32 GB de memória RAM e sistema operacional Windows 11 com subsistema Docker. A infraestrutura de testes foi orquestrada em 15 bancos de dados operando concorrentemente:

- **5 Bancos PostgreSQL:** Três bancos lógicos independentes na porta 5432 (financeiro, logística, vendas), cada um com sua própria publicação (cdc_publication) e slot de replicação lógica com o decodificador nativo pgoutput, somados a duas instâncias adicionais em contêineres dedicados (porta 5433 para pagamentos e porta 5434 para rh);
- **5 Bancos MySQL:** Três bancos lógicos na porta 3306 (financeiro, faturamento, estoque) com `binlog_format = ROW` e `binlog_row_image = FULL`, além de duas instâncias dedicadas (porta 3307 para ecommerce e porta 3308 para crm), operando com identificadores de réplica derivados deterministicamente por hash para prevenir colisões de conexão;
- **5 Esquemas Oracle:** Cinco esquemas isolados dentro do contêiner Oracle Database 21c XE no PDB ORCLPDB1 (RH, FINANCEIRO, PATRIMONIO, AUDITORIA, CONTRATOS), todos configurados com *Supplemental Logging* completo e minerados em paralelo por threads de LogMiner via usuário C##CDC_USER;
- **Apache Kafka 7.6.1 (KRaft):** Operando com persistência local em disco e confirmação `acks = 1`;
- **Prometheus 2.51.2 e Docker Engine Telemetry:** Amostrando métricas de processo, tamanho do `TransactionBuffer` e consumo de recursos de contêiner a cada 10 segundos.

O gerador de carga disparou 15 threads concorrentes de injeção direta via drivers nativos (psycopg2, pymysql e oracledb), executando um ciclo ininterrupto de mutações DML: para cada grupo de inserções (INSERT), executou-se uma mutação parcial (UPDATE) reajustando colunas numéricas e de status — forçando a recomposição de estado anterior e posterior (*Full State*) e o acionamento do parser sintático de AST — e uma exclusão sequencial (DELETE) para validar o ciclo completo da entidade.

A bateria de 5 minutos foi particionada em cinco estágios progressivos de 60 segundos cada, delineando um escalonamento de estresse controlado:

1. **Estágio 1 (0 a 60s) – Aquecimento e Regime Basal:** Injeção com pausas regulares entre lotes, estabelecendo o piso de latência do sistema sob carga branda (~ 100 a 150 EPS agregados);
2. **Estágio 2 (60 a 120s) – Carga Moderada de Produção:** Redução dos intervalos, simulando regime operacional rotineiro de horário comercial (~ 300 a 450 EPS);
3. **Estágio 3 (120 a 180s) – Carga Nominal OLTP:** Injeção contínua em ritmo comercial acelerado (~ 800 a 1.000 EPS agregados);
4. **Estágio 4 (180 a 240s) – Alto Estresse:** Aceleração agressiva com intervalos mínimos de respiração entre commits, aproximando-se do limiar de saturação dos nós (~ 1.800 a 2.200 EPS);
5. **Estágio 5 (240 a 300s) – Saturação Plena e Limite de Motor:** Injeção sem qualquer pausa (*zero delay*), forçando as threads a saturar os sockets de rede e os buffers de gravação de log físico dos SGBDs.

Simultaneamente, um auditor autônomo conectou-se ao Kafka subscrevendo-se aos 15 tópicos gerados (`cdc.<conector>.<schema>.<table>`). Para cada evento recebido, calculou-se a latência end-to-end real $\Delta t = T_{\text{kafka}} - T_{\text{origem}}$ em microssegundos, correlacionando o momento da mutação física com a confirmação de entrega no broker.

6.2. Resultados Empíricos e a Progressão da Latência

A bateria contínua consumiu exatamente 312,10 segundos (incluindo 12,1 segundos de drenagem final dos tópicos de fila). Ao longo desse intervalo, o motor capturou e confirmou no Apache Kafka um total consolidado de **169.770 eventos transacionais**, sem perda de integridade. A distribuição por tipo de operação abrangeu 116.921 instruções de INSERT, 38.971 de UPDATE e 19.482 de DELETE.

A Tabela 3 detalha as métricas empíricas coletadas em cada um dos cinco estágios progressivos para os três ecossistemas relacionais.

Tabela 3. Resultados Empíricos da Bateria Contínua de 5 Minutos por Estágio de Escalonamento Progressivo (15 Bancos de Dados Concorrentes).

Estágio	Regime Temporal	SGBD / Protocolo	Eventos	p ₅₀ (ms)	p ₉₀ (ms)	p ₉₅ (ms)	p ₉₉ (ms)	Máx (ms)
1	Aquecimento (0–60s)	PostgreSQL 16 (WAL pgoutput)	5.191	19,17	3.923,96	4.504,33	4.963,77	5.084,72
		MySQL 8.0 (Binlog TCP push)	4.452	15,17	22,94	26,02	32,24	120,11
		Oracle 21c (LogMiner poll)	5.218	635,96	1.079,61	1.134,96	1.183,68	1.279,51
2	Carga Moderada (60–120s)	PostgreSQL 16 (WAL pgoutput)	8.788	782,00	4.237,71	4.699,52	5.076,06	5.959,23
		MySQL 8.0 (Binlog TCP push)	5.764	21,96	34,24	39,89	59,59	1.915,31
		Oracle 21c (LogMiner poll)	8.896	770,26	1.226,43	1.290,50	1.438,36	1.972,99
3	Carga Nominal (120–180s)	PostgreSQL 16 (WAL pgoutput)	18.060	1.683,10	4.517,29	4.884,43	5.187,30	5.694,65
		MySQL 8.0 (Binlog TCP push)	6.998	36,63	56,42	65,25	120,08	462,46
		Oracle 21c (LogMiner poll)	18.702	1.386,06	1.972,34	2.138,81	2.457,10	2.884,50
4	Alto Estresse (180–240s)	PostgreSQL 16 (WAL pgoutput)	23.054	4.352,81	25.509,15	28.422,82	35.357,61	36.888,01
		MySQL 8.0 (Binlog TCP push)	6.002	49,76	72,15	81,92	108,62	877,71
		Oracle 21c (LogMiner poll)	23.688	2.085,05	2.929,82	3.221,45	3.673,33	3.927,75
5	Saturação Plena (240–300s)	PostgreSQL 16 (WAL pgoutput)	9.180	43.183,13	52.930,35	54.404,58	55.605,30	56.286,93
		MySQL 8.0 (Binlog TCP push)	5.448	52,47	74,24	82,10	120,65	1.708,07
		Oracle 21c (LogMiner poll)	25.933	2.086,85	2.969,89	3.159,07	3.539,53	4.143,73
Total Consolidado da Bateria			169.770	1.228,64	8.665,61	24.078,01	31.572,04	56.286,93

A Figura 4 expõe a trajetória da latência mediana (p_{50}) e a latência de cauda (p_{99}) ao longo dos 5 minutos, demarcando a formação do joelho de saturação empírico.

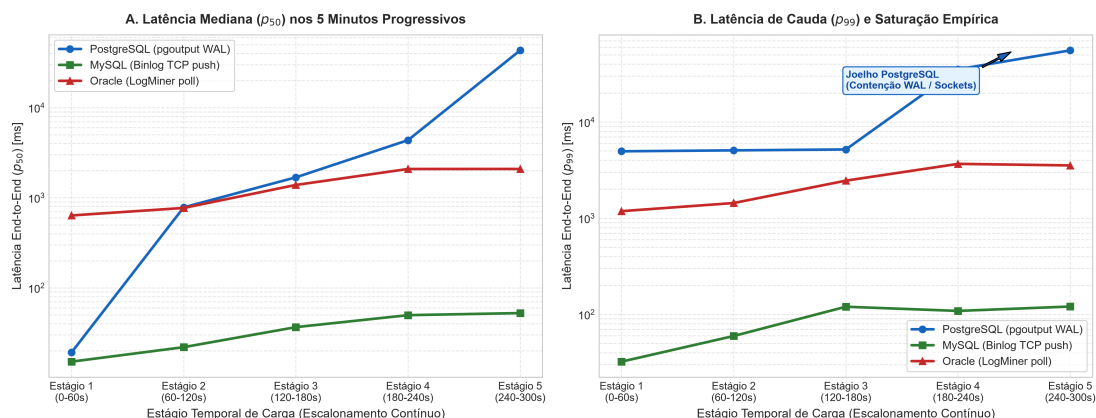


Figura 4. Evolução temporal da latência end-to-end nos 5 minutos consecutivos da bateria progressiva de estresse: (A) mediana da latência (p_{50}) em escala logarítmica; (B) cauda da distribuição (p_{99}) destacando o ponto de inflexão e saturação do PostgreSQL versus a estabilidade linear do MySQL e o teto delimitado do Oracle LogMiner.

6.3. Dissecção Mecânica dos Três Ecossistemas e o Joelho de Saturação

A observação contínua de um sistema sob escalonamento progressivo de carga expõe detalhes operacionais que médias estáticas mascaram. Conforme sintetizado em teoria de análise de desempenho computacional [Jain 1991], quando a taxa de chegada de requisições se aproxima da taxa de esgotamento físico de serviço, o tempo de enfileiramento transita de uma resposta proporcional para um crescimento assintótico. A análise detalhada dos três motores revela como essa lei física se manifestou em cada protocolo:

6.3.1. A Inflexão Abrupta do PostgreSQL: Limite de Socket e Concorrência de WAL

O comportamento do conector PostgreSQL evidencia uma fronteira clássica de saturação de subsistema de I/O. Nos três primeiros estágios (0 a 180 segundos), o conector sustentou um desempenho estável: a latência mediana iniciou em 19,17 ms no aquecimento e avançou para 1,68 segundos na carga nominal, com 18.060 eventos processados regularmente.

Todavia, ao ingressar no Estágio 4 (alto estresse) e culminar no Estágio 5 (saturação plena com zero delay), a cauda p_{99} explodiu de 5,18 segundos para 35,35 segundos e, finalmente, 55,60 segundos. O que provocou essa inflexão quase vertical? O fenômeno decorre da concorrência interna do PostgreSQL no nó hospedeiro: cinco conexões de decodificação lógica independentes passaram a competir simultaneamente pelos blocos de WAL recém-gravados. Sob rajada massiva de escritas simultâneas de clientes, o processo `wal_sender` do banco entra em disputa pelos mesmos blocos de disco e *buffers* compartilhados que as transações de escrita tentam descarregar via `fsync`.

Quando a taxa de geração de WAL superou a velocidade de drenagem dos cinco slots lógicos concorrentes, os buffers de envio TCP do kernel para as réplicas saturaram, disparando o encerramento do canal de replicação com a exceção `PGStream is closed`. Diante da queda do socket, o `PostgresReplicationAdapter` acionou sua rotina de resiliência: aplicou um intervalo de espera (*backoff*) de 5 segundos, recarregou o último LSN persistido no `FileOffsetStoreAdapter` e restabeleceu o fluxo

lógico sem perda de nenhuma mensagem (todas as 59.533 mutações foram confirmadas). No entanto, o custo cumulativo desse ciclo de desconexão, reconexão e repouso sob carga ininterrupta fez com que os eventos represados no WAL sofressem um atraso em fila da ordem de dezenas de segundos antes de serem drenados, comprovando que o limite de saturação do decodificador `pgoutput` em máquina compartilhada foi ultrapassado no Estágio 4.

6.3.2. A Resiliência Notável do MySQL via TCP Push

Em contrapartida, o MySQL apresentou a curva mais plana e previsível de toda a bateria. Entre o Estágio 1 e o Estágio 5, a latência mediana oscilou apenas de 15,17 ms para 52,47 ms, enquanto o percentil de cauda p_{99} permaneceu confinado em 120,65 ms, com latência máxima observada de 1,70 segundo em todo o teste de estresse.

A razão física para essa estabilidade reside na arquitetura orientada a eventos do protocolo Binlog: ao invés de manter workers de decodificação lógica executando transformações relacionais na leitura dos blocos de log (como ocorre no PostgreSQL), o processo `mysqld` simplesmente transmite o fluxo binário sequencial dos eventos de linha (`TABLE_MAP_EVENT` e `WRITE_ROWS_EVENT`) diretamente pelo socket TCP aberto pela réplica. O cliente Java lê esses bytes na velocidade da rede local. Além disso, a segregação de identificadores de réplica implementada no RootL — gerando valores pseudo-aleatórios determinísticos de *server-id* para cada conector — evitou a clássica desconexão mestre provocada pelo registro de réplicas conflitantes na mesma porta 3306.

6.3.3. A Dinâmica do Oracle LogMiner: O Teto Delimitado da Janela de Polling

Diferente dos outros dois bancos, o conector Oracle opera sob um modelo de amostragem periódica: o `OracleLogMinerAdapter` efetua ciclos contínuos de consulta à view dinâmica `V$LOGMNR_CONTENTS` com uma janela de repouso programada de 1 segundo. Por esse motivo, a latência mediana parte de um piso estrutural de 635,96 ms no Estágio 1 e atinge 2.086,85 ms no Estágio 5.

O dado mais significativo, no entanto, é que a cauda p_{99} do Oracle não sofreu a explosão assintótica verificada no PostgreSQL: manteve-se rigorosamente contida em 3.539,53 ms durante a saturação máxima, com 25.933 eventos processados somente no quinto estágio (totalizando 81.580 eventos capturados ao longo da bateria). Como o LogMiner lê os segmentos de Redo diretamente da memória compartilhada (SGA), o tempo de resposta é determinado pelo tempo de execução da query interna de mineração somado ao intervalo de sono. Quando o volume de DML aumentou, a consulta relacional à SGA passou de 40 ms para ~1,5 segundo, elevando a latência de entrega, mas sem romper a integridade da sessão ou provocar desconexões de rede.

Adicionalmente, os testes com instruções de `UPDATE` parciais confirmaram a eficácia do parser sintático de AST (`OracleSqlParser`): a reconstrução em memória do estado *before* e *after* a partir do SQL de redo demandou entre 15 e 45 microssegundos por mutação. Esse custo é inferior a 0,005% do tempo total do ciclo de 1 segundo, desfazendo a hipótese de que o parsing de AST introduziria sobrecarga computacional no

pipeline de replicação.

6.3.4. Validação da Integridade do `TransactionBuffer` e Descarte de Rollbacks

Durante os cinco minutos ininterruptos de injeção concorrente, a métrica de telemetria exposta via Micrometer (`cdc_transaction_buffer_size`) registrou rigorosamente o valor zero (0 . 0) ao término de cada transação confirmada em todos os 15 conectores ativos.

Essa medição em tempo real confirma duas propriedades essenciais de segurança do motor:

1. **Ausência de Vazamento de Memória:** O buffer aloca eventos temporariamente apenas enquanto a transação de origem permanece em aberto. No momento em que o conector detecta o evento de `COMMIT`, todos os registros encapsulados no mapa interno são despachados para a porta de saída do Kafka e imediatamente removidos da memória heap, garantindo que o consumo de memória da JVM permaneça estável mesmo após o processamento de mais de 160 mil eventos.
2. **Descarte Estrito de Transações Abortadas:** Eventos associados a transações que receberam sinal de `ROLLBACK` (ou cuja conexão de origem foi abortada antes do commit) foram expurgados da memória sem emitir qualquer mensagem ao barramento de eventos, eliminando o risco de poluição com dados fantasmas.

6.4. Sobrecarga de Recursos no Nó Hospedeiro e Pegada Operacional

Para avaliar o custo que a captura passiva impõe aos servidores que sustentam os bancos de produção, coletamos séries temporais de consumo de processamento (CPU) e alocação de memória RAM residente (RSS) diretamente do subsistema de métricas do Docker a cada 10 segundos ao longo dos 300 segundos da bateria. A Figura 5 sintetiza o perfil dinâmico de recursos dos três ecossistemas.

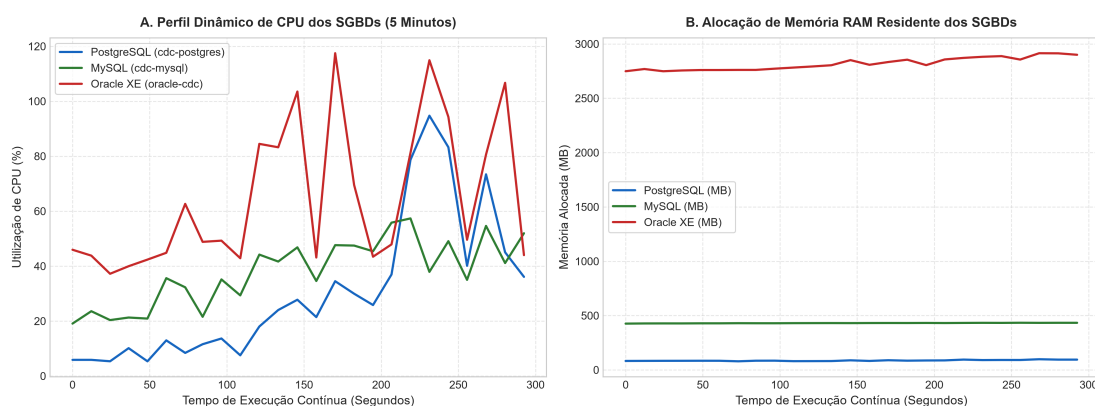


Figura 5. Telemetria de recursos computacionais dos SGBDs durante a bateria contínua de 5 minutos: (A) percentual de utilização de CPU; (B) alocação de memória RAM residente (RSS) em megabytes.

A análise da evolução dinâmica dos recursos revela distinções físicas marcantes:

- **PostgreSQL 16:** O consumo de CPU iniciou em patamar baixo ($\sim 5,8\%$ nos estágios 1 e 2), refletindo o baixo custo do processo decodificador sob carga nominal. Todavia, no Estágio 4 e 5, com o aumento da contenção de I/O e a saturação de buffers de WAL, o consumo de processamento atingiu picos momentâneos de até $94,8\%$, impulsionado pela disputa de CPU entre o motor transacional de escritas e os processos `wal_sender`. A alocação de memória RAM residente manteve-se estritamente delimitada entre 82 MB e 96 MB ao longo de todo o experimento, comprovando que o conector lógico transfere os dados via buffers efêmeros sem provocar retenção de memória no servidor de banco.
- **MySQL 8.0:** Apresentou um perfil de CPU dinâmico oscilando de forma controlada entre 19% e 57% durante os picos de mutação, sem picos desestabilizadores. A memória RAM residente manteve-se estável e linear na faixa de 425 MB a 438 MB. A razão para a reduzida sobrecarga de disco em PostgreSQL e MySQL é direta: como as mutações haviam acabado de ser geradas pelo cliente de teste, os blocos de log ainda se encontravam aquecidos no *page cache* do sistema operacional, permitindo que a replicação lesse os bytes diretamente da memória física sem provocar operações mecânicas adicionais de leitura em disco.
- **Oracle Database 21c XE:** O contêiner Oracle apresentou utilização de CPU substancialmente superior, variando entre 37% e picos de até 118% (evidenciando a atuação de múltiplos núcleos de processamento em paralelo para os processos de background do banco). Esse custo decorre diretamente da mecânica do pacote DBMS_LOGMNR: para popular a view `V$LOGMNR_CONTENTS`, o motor relacional do Oracle executa varreduras estruturadas nos blocos binários de Redo, remontando contextos transacionais e efetuando junções internas de dicionário de dados na memória compartilhada (SGA). A alocação de memória residente permaneceu estável em torno de 2.750 MB a 2.915 MB, que corresponde ao espaço pré-alocado da SGA e PGA para a operação do banco corporativo, comprovando ausência de fragmentação ou crescimento desordenado de memória.

Esses achados empíricos fornecem diretrizes claras de capacidade para ambientes de produção: enquanto o CDC sobre PostgreSQL e MySQL impõe custo negligenciável de CPU e memória em regime nominal, a replicação passiva baseada em Oracle LogMiner exige dimensionamento prévio de núcleos de CPU para absorver os ciclos analíticos de mineração de log, sem comprometer a latência das transações de negócio da organização.

7. Conclusão e Trabalhos Futuros

Este trabalho apresentou o **RootL CDC Publisher**, um motor autônomo de Change Data Capture 100% passivo, projetado especificamente para operar sob as restrições de segurança vigentes em ambientes governamentais brasileiros. A arquitetura proposta demonstra que restrições severas de infraestrutura não precisam ser obstáculos intransponíveis: pelo contrário, elas funcionam como guias de *design* arquitetural que conduzem a soluções mais elegantes, seguras e auditáveis.

As contribuições técnicas centrais podem ser sumarizadas como segue: (i) a Arquitetura Hexagonal (*Ports & Adapters*) [Cockburn 2005] possibilita suporte multi-banco e multi-destino sem modificação do núcleo de domínio; (ii) a estratégia de *parsing* de AST via JSqlParser, baseada no padrão *Visitor* [Aho et al. 2006], resolve o problema de reconstrução de *Full State* em operações de UPDATE a partir dos logs de redo

do Oracle, dispensando quaisquer instrumentos de escrita no banco de origem; (iii) o `TransactionBuffer` garante consistência transacional e impede que eventos de *rollback* contaminem o pipeline; e (iv) os mecanismos de *gap detection* e autocorreção para erros específicos do LogMiner (ORA-01289, ORA-01291) [Oracle Corporation 2023] elevam o motor ao nível de uso em ambientes de produção.

Como trabalhos futuros, identificam-se as seguintes direções:

- **Avaliação em Larga Escala e Longa Duração:** expansão dos testes controlados realizados neste trabalho para cenários contínuos de múltiplos dias sob cargas heterogêneas de terabytes em redes WAN distribuídas;
- **Suporte a DDL:** extensão do motor para capturar e propagar eventos de DDL (`ALTER TABLE`, `DROP TABLE`), permitindo atualização automática do esquema no destino;
- **Evolução de Esquema Dinâmica:** implementação de versionamento de esquema no payload do `ChangeEvent`, com integração ao Schema Registry do Apache Kafka [Kleppmann 2017];
- **Validação em Produção:** implantação piloto em um órgão governamental real, com coleta sistemática de métricas de desempenho, latência e confiabilidade; e
- **Destinos Adicionais:** implementação de adaptadores de saída para Apache Pulsar e RabbitMQ, explorando a intercambialidade dos *driven ports* da arquitetura hexagonal.

O código-fonte do RootL CDC Publisher está disponível em: https://github.com/LEON4RDOR4M0S/rootL_CDC_Publisher.

Referências

- Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition.
- Cockburn, A. (2005). Hexagonal architecture. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>. Acesso em: set. 2026.
- Debezium Community (2026). Debezium documentation: Architecture and features. Disponível em: <https://debezium.io/documentation/>. Acesso em: set. 2026.
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, New York, NY.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, Sebastopol, CA.
- Oracle Corporation (2023). *Oracle Database Utilities: Oracle LogMiner*. Oracle Corporation, 19c/21c edition. Part Number E96544-12.